

Contents

fetch_ch01-24.ipynb —UN Comtrade 一括取得ツール	1
1. セットアップ	1
2. 使い方	2
3. 出力	2
4. 内部で何をしているか	3
5. 設計思想	3
6. その他の設計判断	4
7. 検証済みの事実	5
8. トラブルシューティング	6
付録: 同ディレクトリの <code>comtrade_start.ipynb</code>	6

fetch_ch01-24.ipynb —UN Comtrade 一括取得ツール

UN Comtrade API から HS 6 桁の貿易統計を一括取得するノートブック。既定では **HS 1~24 類（食料・農水産物） × 2012~2025 年 × 全報告国 × 輸出** を取得する。

このドキュメントは、**使い方・内部で何をしているか・なぜその設計にしたか**をまとめたもの。設計判断の根拠はすべて実測値に基づく（数値は本文中に明記）。

1. セットアップ

必要なもの

Python	3.9 以上（動作確認は 3.11 / Anaconda）
ライブラリ	<code>comtradeapicall pandas python-dotenv jupyterlab</code>
API キー	UN Comtrade の Premium サブスクリプションキー （32 文字）

```
pip install comtradeapicall pandas python-dotenv jupyterlab
```

API キーの取得

1. <https://comtradedeveloper.un.org/> でアカウント登録
2. **Products** から Premium 系のプランを Subscribe
3. **Profile** に表示される Primary key / Secondary key（機能差なし、ローテーション用）

無料枠では動きません。無料の preview 系 API は 1 リクエスト 500 レコード・1 期間のみで、このノートブックが使う `getFinalData` / `getCountFinalData` は 401 を返す。所属機関が UN Comtrade の機関契約を持っている場合はそちら経由で取得できることがある。

キーの登録

キーはノートブックに直接書かない（`.ipynb` は出力ごと保存されるため、共有時に漏れる）。ホームディレクトリの外部ファイルから読み込む。

```
read -rs "key?Primary key を貼り付けて Enter: " \
&& printf 'export COMTRADE_KEY=%s\n' "$key" > ~/.comtrade_env \
&& chmod 600 ~/.comtrade_env && unset key && echo " 保存しました"
```

`read -rs` を使う理由は 3 つ。入力が画面に表示されない / 引数ではなく標準入力なのでシェル履歴に残らない / `unset` で変数からも消える。

確認：

```
source ~/.comtrade_env && echo " 長さ: ${#COMTRADE_KEY} 文字"    # → 32 文字
```

ノートブックは python-dotenv でこのファイルを読む (export 接頭辞は自動で解釈される)。

リポジトリ管理下に置く場合、~/.comtrade_env と data/ を必ず .gitignore に入れること。

2. 使い方

```
cd <このディレクトリ> && jupyter lab fetch_ch01-24.ipynb
```

設定セル (セクション 2) だけ書き換えて、上から順に実行する。他のセルは触らなくてよい。

```
YEARS      = [str(y) for y in range(2012, 2026)]    # 2012~2025 年
FLOW       = "X"                                    # 輸出のみ
CHAPTERS   = [f"{i:02d}" for i in range(1, 25)]    # 1~24 類
PARTNER    = "0"                                    # World (全世界合計) のみ
```

変数	意味	書き方
YEARS	対象年のリスト	["2025"] / [str(y) for y in range(2000, 2026)]
CHAPTERS	対象の類 (2 桁ゼロ埋め)	["01"] / ["01", "02"] / [f"{i:02d}" for i in range(1, 25)]
FLOW	貿易フロー	"X" 輸出 / "M" 輸入
PARTNER	相手国	"0" World のみ / None 全相手国

出力先は YEARS と CHAPTERS から自動生成されるので、条件を変えても既存データと混ざらない。

```
data/ex_ch01-24_2012-2025/    ← 1~24類・2012~2025年
data/ex_ch01_2025/           ← 第1類・2025年のみ
```

中断と再開

セル 5 をもう一度実行するだけ。保存済みブロックは自動でスキップされ、続きから走る。カーネルを再起動した場合はセル 1~4 を先に実行すること。

特定のブロックを取り直すときは、該当ファイルを消してから再実行する。

```
rm data/ex_ch01-24_2012-2025/ch03_2018.csv.gz
```

規模の目安 (実測ベース)

設定	件数	時間	容量 (gzip)
1~24 類 / 2025 年 / World	約 89,500	約 3 分	2MB
1~24 類 / 2012~2025 年 / World	約 120 万	約 40~60 分	30MB
1~24 類 / 2025 年 / 全相手国	約 83 万	約 21 分	20MB
全類 / 2000~2025 年 / 全相手国	約 1.9 億	数日	20~30GB

最後の行は API 経由では非現実的。その規模が必要ならバルクダウンロード契約を検討する。

3. 出力

ch{類}_{年}.csv.gz の粒度で保存される。全ファイルを結合した ex*_all.csv.gz も生成される。

列 (18 列)

列	内容
refYear / period	対象年
reporterCode / reporterDesc	報告国 (この統計を報告した国)
flowDesc	Export / Import
partnerCode / partnerDesc	相手国。0 / World は全世界合計
classificationCode	報告に使われた HS 版 (H0=1992 …H6=2022)
cmdCode / cmdDesc	HS 品目コードと品名
qty / qtyUnitAbbr / altQty	数量と単位
netWgt / grossWgt	純重量 / 総重量 (kg)
cifvalue / fobvalue	CIF 価額 / FOB 価額 (USD)
primaryValue	主価額 (USD)。輸出なら FOB、輸入なら CIF

並び順

各ファイル内は **cmdCode** 昇順 → **reporterCode** → **partnerCode**。結合ファイルは 年 → **cmdCode** → 報告国 → 相手国。

API の返却順は報告国コード順で、かつ保証されていないため自前で固定している。順序に依存する処理は書かないこと。

読み込み時の注意

```
df = pd.read_csv(path, dtype={"cmdCode": str, "classificationCode": str})
```

dtype の指定は必須。指定しないと 0201 が 201 になり、先頭ゼロが落ちて品目コードとして使えなくなる。

数量単位の "N/A" (数量未報告を表す API の値) を文字列のまま残したい場合：

```
df = pd.read_csv(path, dtype={"cmdCode": str}, keep_default_na=False, na_values=[""])
```

既定では pandas が "N/A" を欠損として解釈するため、「単位不明」と「欠損」が区別できなくなる。

4. 内部で何をしているか

- (1) HS コード表を取得 `getReference("cmd:HS")` → 8,262件
| `aggrLevel==6` かつ CHAPTERS に該当するものを抽出
- (2) 対象コードを確定 1~24類の6桁 = 1,176件
| 年 × 類 のブロックに分割 (14年 × 24類 = 336ブロック)
- (3) ブロックごとに件数照会 `getCountFinalData` → 真の件数
| 上限を超えるならコード列を再帰的に二分割
- (4) 取得 `getFinalData`
| 件数照会の値と突き合わせ、欠落があれば例外
- (5) 18列に絞りソートして保存 `ch{類}_{年}.csv.gz`

5. 設計思想

このツールの複雑さの大半は、UN Comtrade API の 3 つの落とし穴に対処するためのもの。いずれも「エラーを出さずに間違ったデータを返す」種類の問題で、素直に書くと気づかないまま壊れる。

5.1 落とし穴 ①: 100,000 件の暗黙の打ち切り

`getFinalData` は `maxRecords` に何を指定しても 1 リクエスト 100,000 件で打ち切る。警告も例外も出ない。

実測 (日本・2022 年・全品目・全パートナー・輸出)：

	件数
maxRecords に指定した値	250,000
getCountFinalData が返す真の件数	177,693
getFinalData が実際に返した件数	100,000 (77,693 件が黙って欠落)

対策—取得前に必ず getCountFinalData で真の件数を確認し、取得後に突き合わせる。不足していれば RuntimeError で停止する。**欠損したデータで分析が進むことを防ぐのが最優先**という判断。

```
if got < expected:
    raise RuntimeError(f" 打ち切り: 取得 {got:,} < 期待 {expected:,}")
```

上限を超えると分かっている場合は、コード列を再帰的に二分割してから取得する。

5.2 落とし穴 ②: URL 2000 文字の上限

cmdCode にコードを並べすぎるとリクエストが弾かれる。

Request URL exceeds maximum allowed length (2000 characters).

第 03 類 (魚介類) は 282 コードあり、cmdCode 文字列だけで 1,973 文字になって実際に失敗した。

対策—CODE_STR_MAX = 1_200 を超えたらコード列を二分割する。1 コード 7 文字なので約 170 コードが目安。件数による分割と同じ再帰関数で処理しており、**文字数と件数の両方が収まるまで分割が続く**。

CODE_STR_MAX を 2000 ではなく 1200 にしているのは、URL には他のパラメータも載るため。

5.3 落とし穴 ③: 内訳行の混入による二重計上

partner2Code / customsCode / motCode を None にすると、**合計行と一緒に内訳行が全部返る**。輸送モード別 (Road / Railway / …)、通関手続別 (Clearance for home use / Outward processing / …)、原産国別の内訳がそれぞれ別行として混ざる。

実測 (0201.10・2022 年・輸入・全報告国) :

指定	行数	重複	報告国 88 に対する World 行
3 つとも None	2,613	2,354	734
customsCode / motCode のみ指定	946	601	237
3 つとも合計コードを指定	551	0	88 [OK]

スペイン 1 国だけで World 行が 12 行返り、そのまま合計すると **4 倍以上に膨れる**。

対策—合計コードを固定値としてハードコードし、設定セルの外に置いた。誤って触れないようにするため。

```
partner2Code="0"          # World
customsCode="C00"         # TOTAL CPC
motCode="0"               # TOTAL MOT
```

さらに検証セルで重複行が 0 であることを確認し、崩れていれば停止する。

6. その他の設計判断

なぜ AG6 ではなく明示的なコード列を送るのか

cmdCode="AG6" は「全 6 桁コード」を意味する集計指定子だが、**類を限定できない**。1~24 類だけが必要なので、getReference("cmd:HS") から aggrLevel==6 を抽出して明示的に列挙している。

なお cmdCode="02" のように 2 桁を指定すると、**その 2 桁の集計行 1 件だけが返り**、配下の 6 桁は取れない。6 桁が欲しければ 6 桁を列挙する必要がある。

なぜ「年 × 類」を単位にするのか

再開の粒度と分割の効きやすさのバランス。年だけを単位にすると 1 ブロックが大きすぎ、コード単位にするとリクエスト数が爆発する。

6 桁コードの分布は極端に偏っている (84 類 = 597 号 に対し 45 類 = 7 号、77 類 = 0 号) ため、類ごとに区切ると自然に負荷が平準化される。

なぜ PARTNER="0" (World のみ) が既定なのか

相手国別の内訳を捨てると **データ量が約 1/9 になる** (第 1 類 2025 年の実測: 10,285 行 → 1,109 行 = 10.8%)。

World 行の値は内訳行の合計と厳密に一致するため、**各国の総輸出額を見るだけなら情報は失われない**。

Angola / 010121 / 2025

World	14,026.92	
Dem. Rep. of the Congo	2,511.09	
Russian Federation	1,749.45	内訳合計 = 14,026.92 ← 一致
USA	9,766.38	

貿易相手の分析が必要なら PARTNER = None に戻す。ただし取得規模が 1 桁増えることと、**World 行と個別相手国を合計すると必ず 2 倍になる**点に注意。

なぜ clCode="HS" (as reported) なのか

各国が報告した時点の HS 版をそのまま使う設定。UN による版変換を経ないため、原データに忠実。

代償として、**版をまたぐとコード体系が変わる**。例えば馬の純血種は:

版	コード
HS1992~2007	010110
HS2012 以降	010121 (010129 に分割)

そのため 2025 年のデータに 010110 は 1 行も存在しない (44 コード送って 34 コードしか返らないのは正常)。classificationCode 列で報告版を判別できるようにしてある。

時系列分析で版の差を吸収したい場合は CL を "H6" (HS2022 固定) 等に変更する。UN 側で変換されるが、変換に伴う誤差が入る。どちらが適切かは研究目的次第。

なぜ失敗時に例外で止めるのか

このツールが扱う API は、**失敗時に例外ではなく None や空 DataFrame や不完全なデータを返す**。「エラーが出なかった=成功した」が成り立たない。

そのため、件数の突き合わせ・列の存在確認・重複チェックをすべて明示的に行い、1 つでも崩れたら RuntimeError で停止する方針にしている。**黙って不完全なデータが下流に流れるより、止まって気づけるほうがよい**という判断。

7. 検証済みの事実

検証項目	結果
分割取得の正しさ	44 コード一括 vs 1 コードずつ 44 回 → 全 47 列・1,109 行が完全一致
重複・欠落	行の集合が完全一致 (片側のみの行は 0 件)
再開機能	保存済みブロックをスキップして 0 秒で完了
ソート順	hs_codes.csv の id 順と一致
列構成	指定 18 列が全て存在

分割ロジックは 1 コードずつ取る素朴な方法と等価であることを実データで確認済み。

8. トラブルシューティング

症状	原因と対処
COMTRADE_KEY が未設定	source ~/.comtrade_env を実行、またはカーネル再起動
401 invalid subscription key	無料枠のキーを使っている。Premium が必要
Request URL exceeds maximum allowed length	CODE_STR_MAX を下げる (既定 1200)
打ち切り: 取得 X < 期待 Y	条件を絞る (CHAPTERS を減らす等)。データを信用しないこと
RuntimeError: 内訳行が混入している	partner2Code / customsCode / motCode が合計コードか確認
品目コードの先頭ゼロが消えた	read_csv に dtype={"cmdCode": str} を指定
取得が遅い / 途中で止まる	PAUSE を増やす。中断してもセル 5 の再実行で続きから走る
学内プロキシ環境	各 API 関数に proxy_url= 引数を渡す

付録: 同ディレクトリの comtrade_start.ipynb

品目を 1 つ 2 つ指定して素早く確認するための汎用ノートブック。用途が異なる。

	comtrade_start	fetch_ch01-24
用途	単発の確認・探索	本番の一括取得
保存列	47 列 (API の全列)	18 列
URL 長対策	なし (第 03 類で失敗する)	あり
国の指定	ISO3・国名・数値コードのいずれも可	数値コードのみ

国コードや HS コードを調べる用途にも使える (getReference のラッパーを含む)。